

BioNix_Week_6_Wesley

Tuesday, 12 August 2025 18:35

- The question I ask in weekly meeting
 - o I understand that "nix-env -G <n>" can restore any previous generation. How does this compare in reliability with using "nix-shell" + pinned channels for reproducible environments? Is one preferred for research workflows at WEHI?
 - o A: is really need to depend the environment, if in the very simple environment, -G is easy to jump between different environments, which they are not interact between each layers. However, in very complex environment, which might earlier to cause the errors. The -rollback might be more suitable in more complexity environment; it's more like "if the iPhone do the upgrade, and some errors appears, then de-upgrade, back to previous version"
- The key things in the meeting
 - o Use the "man" code to check the whole pages
 - o Cuz the pkgs/files always run in the latest version, if in the interapted environment, try to "pin" the version run in the initial, avoiding the error

Closures

- Closure = complete set of dependencies of a derivation (direct + indirect).
- Command: `nix-store -qR $(which man)` → lists full closure of man.
- Copying closure to another machine → run software immediately (basis of deployment).
- `nix-store -q --tree <path>` → shows tree of dependencies, helps trace why a dependency exists.
- Applies to environments too: `nix-store -q --tree ~/.nix-profile`.

```
MacBookPro:~ wesley$ nix-store -qR "$(which man)"
error: path '/usr/bin/man' is not in the Nix store
```

```
fix
```

The code is not finding, the previous env is not satisfies the running code

1. Install the man nix pkg
2. Run which man
3. Processing nix-store

```
MacBookPro:~ wesley$ nix-env -iA nixpkgs.man-db
installing 'man-db-2.13.1'
building '/nix/store/07x1zmi1j1z94zfvk2fq6p9g0s18bxbp6j-user-environment.drv'...
MacBookPro:~ wesley$ which man
/Users/wesley/.nix-profile/bin/man
MacBookPro:~ wesley$ nix-store -qR "$(which man)"
/nix/store/0vychg1lq6j44xzlny1bja7gpcrl2pba-libiconv-1.17
/nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0
/nix/store/18bv9i5gmfx0zig5lw4h5fp713w8m1v-gzip-1.14
/nix/store/1zs1597hmr3ynm0firvlj4n7snj9mv6-libpipeline-1.5.8
/nix/store/z5dq4nh9nynddz1j10sfcd1rqnzlmjq-q-libcxx-19.1.7
/nix/store/khyldgrbi9iii32s67l57908qd373388-zstd-1.5.7
/nix/store/qipw6ppxf5dv6j8klls06q5ixhb77y4-pcre2-10.44
/nix/store/zfxb1786sivl2psmqj41y8r6avvv-gnugrep-3.12
/nix/store/810nlh5qdl7ixk65pi4i3z4w27977jh-zstd-1.5.7-bin
/nix/store/8xi6xc6hgs7saynm4xnlpd199k37ddfi-libiconv-109
/nix/store/afjran1fs9n6xhw8i0rxrgqlgm4jajfq-groff-1.23.0
/nix/store/xfg997s49k0mdd8pfb1699g808hl8pfbk-gdbm-1.25-lib
/nix/store/gxkk8jwnibb0rmh9987h1aaxsspnlibi-man-db-2.13.1
MacBookPro:~ wesley$ nix-store -q --tree "$(which man)"
/nix/store/gxkk8jwnibb0rmh9987h1aaxsspnlibi-man-db-2.13.1
├── /nix/store/0vychg1lq6j44xzlny1bja7gpcrl2pba-libiconv-1.17
│   ├── /nix/store/0vychg1lq6j44xzlny1bja7gpcrl2pba-libiconv-1.17 [...]
│   └── /nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0
│       ├── /nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0 [...]
│       └── /nix/store/18bv9i5gmfx0zig5lw4h5fp713w8m1v-gzip-1.14
│           ├── /nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0 [...]
│           ├── /nix/store/18bv9i5gmfx0zig5lw4h5fp713w8m1v-gzip-1.14 [...]
│           └── /nix/store/1zs1597hmr3ynm0firvlj4n7snj9mv6-libpipeline-1.5.8
│               ├── /nix/store/1zs1597hmr3ynm0firvlj4n7snj9mv6-libpipeline-1.5.8 [...]
│               ├── /nix/store/810nlh5qdl7ixk65pi4i3z4w27977jh-zstd-1.5.7-bin
│               ├── /nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0 [...]
│               ├── /nix/store/z5dq4nh9nynddz1j10sfcd1rqnzlmjq-q-libcxx-19.1.7
│               ├── /nix/store/z5dq4nh9nynddz1j10sfcd1rqnzlmjq-q-libcxx-19.1.7 [...]
│               ├── /nix/store/khyldgrbi9iii32s67l57908qd373388-zstd-1.5.7
│               ├── /nix/store/z5dq4nh9nynddz1j10sfcd1rqnzlmjq-q-libcxx-19.1.7 [...]
│               ├── /nix/store/khyldgrbi9iii32s67l57908qd373388-zstd-1.5.7 [...]
│               ├── /nix/store/zfxb1786sivl2psmqj41y8r6avvv-gnugrep-3.12
│               ├── /nix/store/jyqk60qw4s3xjrqa5k6h4kyc87bg3x4-bash-5.3p0 [...]
│               └── /nix/store/qipw6ppxf5dv6j8klls06q5ixhb77y4-pcre2-10.44
```

```

└─/nix/store/khyldgrb1911i32s67/57908qd3/3388-zstd-1.5.7 [...]
└─/nix/store/zfxb1786sivl2psmqj141y8r6avvw-gnugrep-3.12 [...]
└─/nix/store/jyqk60qw4s3xjrqb5k6h4kyc87bg3x4-bash-5.3p0 [...]
└─/nix/store/qipw6ppxf5dv6j8kills06q5ixhb77y4-pcre2-10.44 [...]
└─/nix/store/qipw6ppxf5dv6j8kills06q5ixhb77y4-pcre2-10.44 [...]
...
└─/nix/store/zfxb1786sivl2psmqj141y8r6avvw-gnugrep-3.12 [...]
└─/nix/store/810inlh5qd17ixk65pi4i3z4w27977jh-zstd-1.5.7-bin [...]
└─/nix/store/afjran1fs9n6xhw8i0rxrgqlgm4jajfq-groff-1.23.0 [...]
└─/nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109 [...]
└─/nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109 [...]
└─/nix/store/jyqk60qw4s3xjrqb5k6h4kyc87bg3x4-bash-5.3p0 [...]
└─/nix/store/z5dq4nh9nynddz1j10sfcd1rqnzlmjq-q-libcxx-19.1.7 [...]
└─/nix/store/afjran1fs9n6xhw8i0rxrgqlgm4jajfq-groff-1.23.0 [...]
└─/nix/store/xfg997s49k0mdd8pfb1699g808hl8pfb-gdbm-1.25-lib [...]
└─/nix/store/xfg997s49k0mdd8pfb1699g808hl8pfb-gdbm-1.25-lib [...]
└─/nix/store/gxkk8jwnibb0rmh9987h1aaxsspnlibi-man-db-2.13.1 [...]

```

Dependency Resolution

- Unlike apt, Nix does not solve version constraints.
- Dependencies are static and explicit: each variant is a new derivation.
- No SAT solver needed; avoids “dependency hell.”

Recovering the Hard Way

- `nix-env -e '*'` removes all packages (even Nix).
- Recovery:
 1. Run Nix binary directly from `/nix/store/...-nix-2.1.3/bin/nix-env`.
 2. Options:
 - `--rollback` → revert to previous environment generation.
 - `-i <path>` → reinstall Nix, creating a new generation.

```

MacBookPro:~ wesley$ nix-env -e '*'
uninstalling 'man-db-2.13.1'
uninstalling 'hello-2.12.2'
uninstalling 'cowsay-3.8.4'
building '/nix/store/xsk9wjy0dpp7w7d3r3bkzxzaklqdzq6q-user-environment.drv'...
MacBookPro:~ wesley$ nix-env -e '*'
uninstalling 'hello-2.10'
-bash: nix-env: command not found
MacBookPro:~ wesley$ nix-env -e '*'
uninstalling 'nix-2.1.3'

```

Channels

- Packages come from *channels* managed by `nix-channel`.
- `nix-channel --list` shows configured channels (e.g., `nixpkgs-unstable`).
- `nix-channel --update` refreshes package definitions (similar to `apt-get update`).
- Channel data stored under `~/nix-channels` and unpacked to `~/nix-defexpr/channels`.

Basics of the Nix Language

Core ideas

- Everything = expression (no statements).
- Values immutable (functional style).
- Nix = pure, lazy, functional DSL.
- Use `nix repl` to test expressions.

```

nix repl (open REPL)
q (quit)
? (help)

```

- Arithmetic / Operators

```

MacBookPro:~ wesley$ nix repl
Nix 2.30.2
Type :? for help.
nix-repl> 1+3
4

```

```

nix-repl> 4_2
error: undefined variable '_2'
at «string»:1:2:
1 | 4_2
  |   ^

```

```

nix-repl> 4-2
2

```

```

nix-repl> 3*3
9

```

```
9
```

```
nix-repl> 4/1  
/Users/wesley/4/1
```

```
nix-repl> 4 /2  
error: attempt to call something which is not a function but an integer. 4  
      at <string>:1:1:  
          1| 4 /2  
            | ^
```

```
nix-repl> 4/ 2  
2
```

- Strings

```
nix-repl> "foo"  
"foo"
```

```
nix-repl> 'foo'  
"foo"  
"foo"      # double quotes  
"foo"      # two single quotes  
"${foo}"   # interpolate expr  
"\${foo}"  # escape
```

- Only "\${...}" works.
- Cannot mix int + string (must convert).

- List

```
nix-repl> [ 2 "foo" true (2+3) ]  
[  
  2  
  "foo"  
  true  
  5  
]
```

- Space separated, not comma.
- Immutable → operations return new list.

- Attribute sets

```
s = { foo = "bar"; a-b = "baz"; "123" = "num"; };  
s.foo    # "bar"  
s.a-b    # "baz"  
s."123"  # "num"
```

```
nix-repl> s = { foo = "bar"; a-b = "baz"; "123" = "num"; }
```

```
nix-repl> s  
{  
  "123" = "num";  
  a-b = "baz";  
  foo = "bar";  
}
```

```
nix-repl> s.a-b  
"baz"
```

```
nix-repl> s."123"  
"num"
```

- Recursive with `rec`

```
nix-repl> rec { a = 3; b = a+4; }  
{  
  a = 3;  
  b = 7;  
}
```

Q: Since everything in Nix is an expression, how do researchers at WEHI usually balance writing raw expressions in .nixfiles versus using higher-level helper functions from nixpkgs?

- I see how everything in Nix is just an expression, but in practice people rarely write raw ones; they reuse helpers. I want to know if BioNix prefers encouraging students to “learn

the bare syntax" first or immediately rely on abstractions in nixpkgs.

Q: Values in Nix are immutable. In practice, how does this immutability help with reproducibility of research workflows compared to more dynamic scripting approaches (e.g., Python virtualenvs)?

- The immutability model seems to directly enforce reproducibility, since nothing can be silently changed once built. I think this contrasts with Python virtualenvs, where dependencies can drift. I want to hear if this property is central to WEHI reproducibility standards.

Q: I noticed Nix only supports interpolation via "\${...}". When writing derivations, do you recommend heavy use of string interpolation for paths and versions, or is it better to rely on attribute sets for clarity?

- It feels powerful to interpolate \${...} for paths and versions, but also messy if overused. Maybe attribute sets (with keys for version numbers, URLs, etc.) are cleaner in the long run. I want to check if there's a BioNix style guideline here.

Q: Recursive attribute sets seem powerful for package definitions. Are there best practices for when to use rec { ... } versus plain sets in BioNix scripts?

- Recursive sets (rec) let me reference inside definitions, but they could make code harder to follow if overused. I'm wondering if the BioNix team has rules-of-thumb (e.g., only use rec for package definitions, not for simple configs).

Q: Both let and with bring variables into scope. At WEHI, is there a coding style preference (e.g., avoid with for clarity), or is it acceptable in derivations and pipeline scripts?

- Both bring variables into scope, but with feels riskier because it pollutes scope. Many languages avoid this pattern for clarity. I'd like to ask if BioNix encourages sticking with let for readability.

Q: Nix laziness means unused expressions aren't evaluated. In a large BioNix workflow, does this practically reduce build time, or are most expressions eventually needed anyway?

- Laziness means unused derivations won't be evaluated. My hunch: in small scripts this saves time, but in large workflows everything is eventually needed, so the gain is limited. Curious if WEHI sees real build-time savings from laziness.